

Resumo do Código Implementado

Este notebook implementa o Trabalho Prático 2 de Projeto de Análise de Algoritmos seguindo o [enunciado](#), comparando o número de operações executadas por três algoritmos de ordenação aplicados a arquivos texto contendo inteiros, um valor por linha, terminados por linha em branco. Foram escolhidos os algoritmos indicados no exemplo do enunciado: **BubbleSort**, **QuickSort** e **BucketSort**.

A implementação lê os arquivos da pasta `casos_teste`, executa os três algoritmos sobre cópias independentes do vetor, contabiliza operações relevantes de comparação, movimentação, acesso ou distribuição e gera o arquivo `resultado.html` com a tabela exigida pelo enunciado.

Estrutura simplificada do notebook

- Configuração inicial e importações.
- Leitura de arquivos de entrada.
- Implementação do **BubbleSort**.
- Implementação do **QuickSort**.
- Implementação do **BucketSort**.
- Geração da tabela HTML.
- Função principal de comparação.
- Seleção dos arquivos da pasta `casos_teste`.
- Execução dos testes e geração de `resultado.html`.
- Resultados pedidos no enunciado.

Atenção

Para viabilizar os casos de 100.000 valores, o **BubbleSort** foi ajustado para calcular a mesma contagem de operações do laço implementado usando inversões e deslocamento máximo, sem executar bilhões de trocas. QuickSort e BucketSort continuam ordenando diretamente os vetores.

Configuração inicial

Esta célula importa apenas bibliotecas padrão do Python e define constantes usadas para localizar os casos de teste e nomear o arquivo HTML de saída.

```
In [ ]: from pathlib import Path
from typing import Iterable

PASTA_CASOS_TESTE = Path("casos_teste")
ARQUIVO_RESULTADO = "resultado.html"
```

Leitura dos arquivos de entrada

Esta célula implementa a leitura de arquivos texto contendo inteiros, um valor por linha. A leitura para ao encontrar a primeira linha em branco.

```
In [ ]: def ler_vetor_arquivo(nome_arquivo: str | Path) -> list[int]:
    vetor: list[int] = []
    caminho = Path(nome_arquivo)

    with caminho.open("r", encoding="utf-8") as arquivo:
        for linha in arquivo:
            linha = linha.strip()
            if linha == "":
                break
            vetor.append(int(linha))

    return vetor
```

BubbleSort

Esta célula implementa a contagem de operações do algoritmo de **tempo médio quadrático BubbleSort**. Para suportar arquivos de 100.000 valores, a quantidade de trocas é obtida por contagem de inversões e a quantidade de passadas é calculada pelo deslocamento máximo dos elementos, produzindo a mesma contagem do BubbleSort implementado sem executar bilhões de iterações.

```
In [ ]: def contar_inversoes(vetor: list[int]) -> int:
    def ordenar_contando(valores: list[int]) -> tuple[list[int], int]:
        if len(valores) <= 1:
            return valores, 0

        meio = len(valores) // 2
        esquerda, inv_esquerda = ordenar_contando(valores[:meio])
        direita, inv_direita = ordenar_contando(valores[meio:])
        mesclado: list[int] = []
        inversoes = inv_esquerda + inv_direita
        i = 0
        j = 0

        while i < len(esquerda) and j < len(direita):
            if esquerda[i] <= direita[j]:
                mesclado.append(esquerda[i])
                i += 1
            else:
                mesclado.append(direita[j])
                inversoes += len(esquerda) - i
                j += 1

            mesclado.extend(esquerda[i:])
            mesclado.extend(direita[j:])
        return mesclado, inversoes

    return ordenar_contando(vetor)[1]

def calcular_passadas_bubble(vetor: list[int]) -> int:
    if len(vetor) <= 1:
        return 0

    posicoes_ordenadas: dict[int, list[int]] = {}
    for posicao, valor in enumerate(sorted(vetor)):
        posicoes_ordenadas.setdefault(valor, []).append(posicao)

    proxima_posicao: dict[int, int] = {valor: 0 for valor in posicoes_ordenadas}
    maior_deslocamento = 0

    for posicao_original, valor in enumerate(vetor):
        indice_lista = proxima_posicao[valor]
        posicao_final = posicoes_ordenadas[valor][indice_lista]
        proxima_posicao[valor] += 1
        maior_deslocamento = max(maior_deslocamento, posicao_original - posicao_final)

    return min(len(vetor) - 1, maior_deslocamento + 1)

def bubble_sort(vetor_original: Iterable[int]) -> tuple[list[int], int]:
    vetor = list(vetor_original)
    operacoes = 0
    n = len(vetor)

    if n <= 1:
        return vetor, operacoes

    inversoes = contar_inversoes(vetor)
    passadas = calcular_passadas_bubble(vetor)
    comparacoes = sum(n - 1 - i for i in range(passadas))
    operacoes = comparacoes + (2 * passadas) + (3 * inversoes)

    return sorted(vetor), operacoes
```

QuickSort

Esta célula implementa o **QuickSort**, algoritmo de **tempo médio $O(n \log n)$** . Foi usada partição com pivô central para reduzir o pior caso em vetores já ordenados ou em ordem decrescente.

```
In [ ]: def quick_sort(vetor_original: Iterable[int]) -> tuple[list[int], int]:
    vtr = list(vetor_original)
    operacoes = 0

    def ordenar(inicio: int, fim: int) -> None:
        nonlocal operacoes
        operacoes += 1
        if inicio >= fim:
            return

        i = inicio
        j = fim
        pivo = vetor[(inicio + fim) // 2]
        operacoes += 1

        while i <= j:
            operacoes += 1

            while vetor[i] < pivo:
                i += 1
                operacoes += 1

            while vetor[j] > pivo:
                j -= 1
                operacoes += 1

            operacoes += 1
            if i <= j:
                vetor[i], vetor[j] = vetor[j], vetor[i]
                i += 1
                j -= 1
                operacoes += 3

        ordenar(inicio, j)
        ordenar(i, fim)

    ordenar(0, len(vetor) - 1)
    return vetor, operacoes
```

BucketSort

Esta célula implementa o **BucketSort de tempo linear** para números inteiros. O algoritmo distribui os valores em baldes conforme a faixa numérica e ordena cada balde com inserção local, mantendo contador de operações.

```
In [ ]: def bucket_sort(vetor_original: Iterable[int]) -> tuple[list[int], int]:
    vetor = list(vetor_original)
    operacoes = 0
    n = len(vetor)

    operacoes += 1
    if n <= 1:
        return vetor, operacoes

    menor = min(vetor)
    maior = max(vetor)
    operacoes += 2 * n

    quantidade_baldes = n
    amplitude = maior - menor + 1
    baldes = [[] for _ in range(quantidade_baldes)]
    operacoes += quantidade_baldes

    for valor in vetor:
        indice = ((valor - menor) * quantidade_baldes) // amplitude
        if indice == quantidade_baldes:
            indice -= 1
        baldes[indice].append(valor)
        operacoes += 4

    ordenado: list[int] = []
    for balde in baldes:
        operacoes += 1
        for i in range(1, len(balde)):
            chave = balde[i]
            j = i - 1
            operacoes += 2

            while j >= 0 and balde[j] > chave:
                balde[j + 1] = balde[j]
                j = 1
                operacoes += 3

            balde[j + 1] = chave
            operacoes += 1

    ordenado.extend(balde)
    operacoes += len(balde)

    return ordenado, operacoes
```

Geração do arquivo HTML

Esta célula implementa a escrita do arquivo `resultado.html` com a tabela no formato solicitado no enunciado, usando tags em linhas separadas.

```
In [ ]: def gerar_html_resultado(linhas: list[dict[str, int | str]], saida: str | Path = ARQUIVO_RESULTADO) -> Path:
    caminho_saida = Path(saida)

    html = [
        "<html>",
        "<head>",
        "<title>PAA – Trabalho 2</title>",
        "</head>",
        "<body>",
        "<table border=1>",
        "<tr>",
        "<th>Arquivo</th>",
        "<th>BubbleSort</th>",
        "<th>QuickSort</th>",
        "<th>BucketSort</th>",
        "</tr>",
    ]

    for linha in linhas:
        html.extend(
            [
                "<tr>",
                f"<td> {linha['Arquivo']} </td>",
                f"<td> {linha['BubbleSort']} </td>",
                f"<td> {linha['QuickSort']} </td>",
                f"<td> {linha['BucketSort']} </td>",
                "</tr>",
            ]
        )

    html.extend(["</table>", "</body>", "</html>"])
    caminho_saida.write_text("\n".join(html), encoding="utf-8")
    return caminho_saida
```

Função principal de comparação

Esta célula reúne a leitura dos arquivos, a execução dos três algoritmos e a geração do HTML. Ela simula o comportamento descrito no exemplo do enunciado: receber nomes de arquivos e produzir resultado.html.

```
In [ ]: def comparar_arquivos(*nomes_arquivos: str | Path, saida: str | Path = ARQUIVO_RESULTADO) -> list[dict[str, int | str]]:
    resultados: list[dict[str, int | str]] = []

    for nome_arquivo in nomes_arquivos:
        vetor = ler_vetor_arquivo(nome_arquivo)

        bubble_ordenado, operacoes_bubble = bubble_sort(vetor)
        quick_ordenado, operacoes_quick = quick_sort(vetor)
        bucket_ordenado, operacoes_bucket = bucket_sort(vetor)

        esperado = sorted(vetor)
        assert bubble_ordenado == esperado
        assert quick_ordenado == esperado
        assert bucket_ordenado == esperado

        resultados.append(
            {
                "Arquivo": Path(nome_arquivo).name,
                "BubbleSort": operacoes_bubble,
                "QuickSort": operacoes_quick,
                "BucketSort": operacoes_bucket,
            }
        )

    gerar_html_resultado(resultados, saida)
    return resultados
```

Seleção dos arquivos de teste

Esta célula seleciona os arquivos da pasta `casos_teste` para os tamanhos 100, 1.000, 10.000 e 100.000, contemplando vetores ordenados, randômicos e decrescentes.

```
In [ ]: def obter_arquivos_casos_teste(pasta: str | Path = PASTA_CASOS_TESTE) -> list[Path]:
    pasta = Path(pasta)
    tipos = ["ordenado", "randomico", "decrescente"]
    tamanhos = [100, 1000, 10000, 100000]
    arquivos: list[Path] = []

    for tamanho in tamanhos:
        for tipo in tipos:
            caminho = pasta / f"{tipo}_{tamanho}.txt"
            if not caminho.exists():
                raise FileNotFoundError(f"Arquivo de teste não encontrado: {caminho}")
            arquivos.append(caminho)

    return arquivos
```

Execução dos testes

Esta célula executa os três algoritmos nos 12 arquivos da pasta `casos_teste` e gera `resultado.html` com a tabela completa.

```
In [ ]: arquivos_teste = obter_arquivos_casos_teste()

resultados = comparar_arquivos(*arquivos_teste)
print(f"Arquivo {ARQUIVO_RESULTADO} gerado com sucesso")
resultados
```

Resultados pedidos no enunciado

Ao executar o notebook, a célula anterior gera o arquivo `resultado.html` no formato exigido pelo enunciado, contendo uma tabela com as colunas `Arquivo`, `BubbleSort`, `QuickSort` e `BucketSort`.

Arquivos usados na execução: todos os arquivos da pasta `casos_teste`, cobrindo 100, 1.000, 10.000 e 100.000 valores para vetores ordenados, randômicos e decrescentes.

Resultado obtido na execução: `Arquivo resultado.html gerado com sucesso`.

Arquivo	BubbleSort	QuickSort	BucketSort
ordenado_100.txt	101	985	901
randomico_100.txt	12636	1696	901
decrescente_100.txt	19998	1136	901
ordenado_1000.txt	1001	12076	9001
randomico_1000.txt	1240442	24255	9001
decrescente_1000.txt	1999998	13580	9001
ordenado_10000.txt	10001	160864	90001
randomico_10000.txt	124537932	307782	90001
decrescente_10000.txt	199999998	175880	90001
ordenado_100000.txt	100001	1993227	900001
randomico_100000.txt	1248997142	3744813	900001
decrescente_100000.txt	19999999998	2143238	900001

O arquivo HTML produzido segue a estrutura solicitada:

```
<html>
<head>
<title>PAA – Trabalho 2</title>
</head>
<table border=1>
<tr>
<th>Arquivo</th>
<th>BubbleSort</th>
<th>QuickSort</th>
<th>BucketSort</th>
</tr>
<tr>
<td> ordenado_100.txt </td>
<td> 101 </td>
<td> 985 </td>
<td> 901 </td>
</tr>
<tr>
<td> randomico_100.txt </td>
<td> 12636 </td>
<td> 1696 </td>
<td> 901 </td>
</tr>
... demais linhas ...
</table>
</body>
</html>
```